SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1715**

P.L.OAVIKSHA(192572001)

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 20%

Duration: 1 Hour

QUESTIONS: 5 **Total Marks:50**

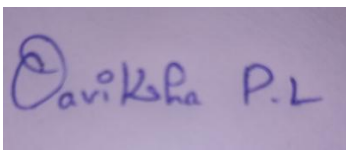
Annexure A

Scenario Based Assignment

Code of Conduct:

I _P.L.OAVIKSHA(192572001) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic iii. Analyze the impact of heuristic accuracy on both algorithms iv. Discuss how dynamic changes (blocked paths) affect search performance

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. Scenario: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

i. Explain how Hill Climbing would work in this scenario and identify its limitations

ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. Scenario: An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

i. Explain why this problem requires an online search agent instead of offline search ii. Analyze the challenges of partial observability and dynamic environments

iii. Describe how the rover builds and updates its internal model iv. Compare online search with uninformed and informed search strategies

v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

i. Clearly define variables, domains, and constraints with explanation

ii. Explain how backtracking search works in this scenario

iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency

iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must complete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

i. Explain how the Minimax algorithm models this problem

ii. Analyze the computational challenges of large game trees

iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning

iv. Design an evaluation function and justify the factors considered

v. Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

QUESTION 1:

i. Greedy Best-First Search (GBFS) Behavior

Greedy Best-First Search selects the path that appears closest to the destination using only the heuristic value **h(n)** (straight-line distance). It ignores the actual travel cost from the starting point.

Strengths

- Very fast decision-making.
- Requires less computation.
- Suitable for urgent emergency response where quick action is required.
- Quickly reaches the estimated goal if the heuristic is good.

Weaknesses

- May choose unsafe or blocked routes because it ignores actual travel cost.
- Cannot guarantee the shortest or safest path.
- Performance decreases when real-time conditions change frequently.
- May require repeated replanning if obstacles appear.

*ii. A Search Behavior**

A* Search evaluates each path using the function:

$$\boxed{f(n) = g(n) + h(n)}$$

Where:

- **g(n)** = Actual travel cost from the start.
- **h(n)** = Estimated remaining distance to the goal.

A* balances the real movement cost and heuristic estimate to find the most efficient route.

Advantages

- Finds the optimal path when the heuristic is admissible.
- Considers weather conditions, flooded roads, and terrain cost.
- Produces safer and more reliable routes.
- Adapts better when movement costs vary.

Limitations

- Requires more computation and memory than GBFS.
- Replanning is needed when the environment changes.

iii. Impact of Heuristic Accuracy

Heuristic Accuracy	**Greedy Best-First Search**	**A* Search**
Highly Accurate	Very fast and usually reaches the goal quickly	Finds the optimal path efficiently
Moderately Accurate	May choose unnecessary detours	Still performs well with slight extra computation
Poor/Inaccurate	Can move toward blocked or unsafe paths	Performance slows but usually still finds a good path
Dynamic Environment	Frequently requires replanning	More stable because actual costs are also considered

Analysis

- Better heuristics improve both algorithms.
- GBFS depends almost entirely on heuristic quality.
- A* is less affected because it also considers actual travel cost.

iv. Effect of Dynamic Changes (Blocked Paths)

In flood-affected regions:

- Roads may suddenly become blocked.
- Weather continuously changes travel cost.
- New obstacles appear unexpectedly.

Greedy Best-First Search

- May continue toward blocked paths.
- Needs frequent route recalculation.
- Can waste time by exploring incorrect paths.

*A Search**

- Updates the path using new cost information.
- Chooses alternative safer routes.
- Maintains better route quality despite environmental changes.
- Although replanning increases computation, it improves reliability.

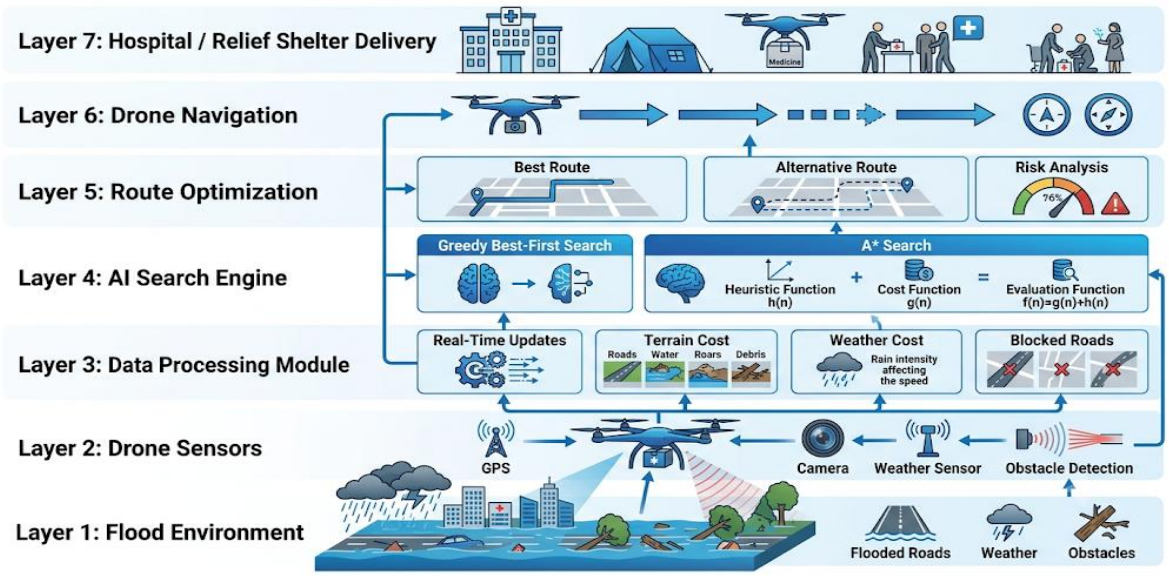
v. Recommended Algorithm

Recommended Algorithm: A* Search

Justification

- Produces the optimal and safest route.
- Considers both actual travel cost and estimated distance.
- Handles varying terrain costs and weather conditions effectively.
- More reliable in dynamic flood environments.
- Reduces the risk of drone failure and medicine delivery delays.

Although Greedy Best-First Search is faster, its inability to consider actual movement cost makes it unsuitable for critical emergency operations where safety and reliability are essential.



QUESTION 2:

i. Explain how Hill Climbing would work in this scenario and identify its limitations.

Working of Hill Climbing

Hill Climbing is a local search optimization algorithm that starts with an initial traffic signal timing and continuously searches for a better neighboring solution.

Steps

1. Start with the current traffic signal timing.
2. Measure traffic performance.
3. Generate neighboring signal timing configurations.
4. Compare the neighboring solution with the current solution.
5. If the neighboring solution improves traffic flow, replace the current solution.
6. Repeat until no better neighboring solution exists.

Advantages

- Fast execution.
- Easy to implement.
- Requires less memory.
- Suitable for quick local optimization.

Limitations

- Gets trapped in local maxima.
- Cannot guarantee the global optimum.
- Sensitive to the starting solution.
- Performs poorly in dynamic traffic environments.

ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation.

1. Local Maxima

A local maximum is a solution that is better than neighboring solutions but is not the best overall solution.

Real-world Interpretation:

The AI optimizes traffic at one intersection but increases congestion at nearby intersections. Since no neighboring solution appears better, the algorithm stops before reaching the global optimum.

2. Plateau

A plateau is a flat region where many neighboring solutions have almost the same evaluation value.

Real-world Interpretation:

Changing the green signal duration by a few seconds does not significantly improve traffic flow. The algorithm cannot determine which direction to move, causing it to stop or wander.

3. Ridge

A ridge is a narrow path where better solutions require multiple coordinated changes instead of a single change.

Real-world Interpretation:

Traffic congestion can only be reduced by changing signal timings at several intersections simultaneously. Hill Climbing changes one signal at a time, making it difficult to reach the optimal solution.

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations.

Simulated Annealing

Simulated Annealing improves Hill Climbing by occasionally accepting worse solutions during the search. This helps the algorithm escape local maxima and continue exploring better solutions.

Advantages

- Escapes local maxima.
- Explores a larger search space.
- Finds better global solutions.
- Suitable for dynamic traffic conditions.

Genetic Algorithm

A Genetic Algorithm is inspired by natural evolution. Each traffic signal timing configuration is treated as a chromosome.

The algorithm performs:

- **Selection** – Chooses the best solutions.
- **Crossover** – Combines good solutions to create new ones.
- **Mutation** – Introduces random changes to explore new possibilities.

Advantages

- Searches multiple solutions simultaneously.
- Escapes local maxima.
- Handles very large search spaces.
- Produces near-optimal solutions.
- Adapts quickly to changing traffic patterns.

iv. Recommend the best approach and justify your choice based on scalability and adaptability.

Recommended Algorithm: Genetic Algorithm

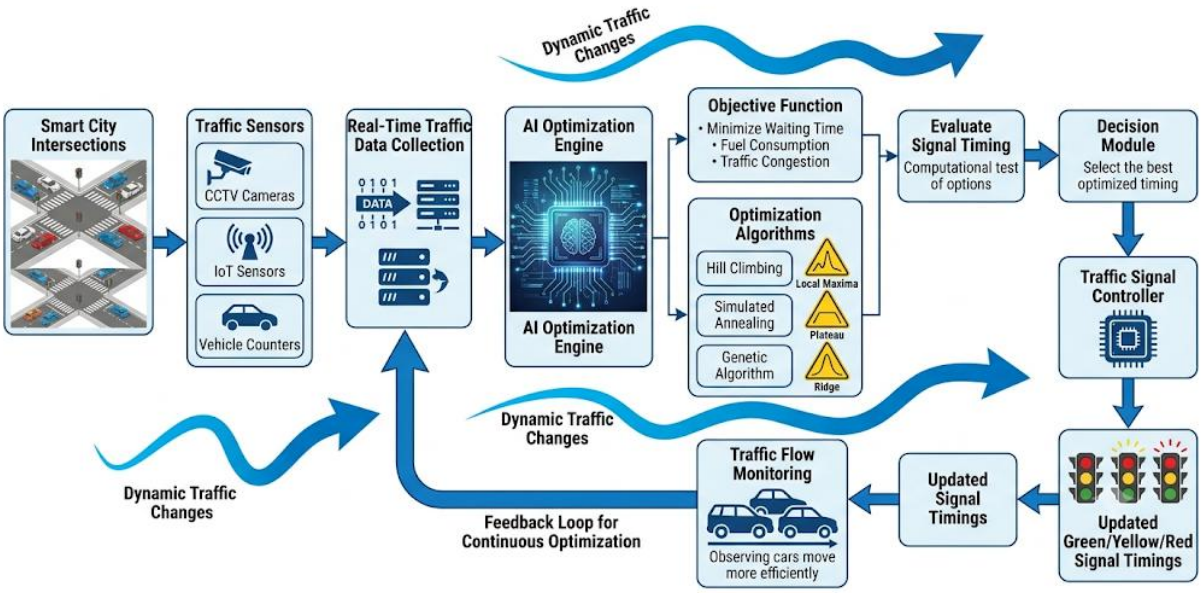
Justification

The **Genetic Algorithm** is the most suitable approach because:

- Optimizes hundreds of traffic intersections simultaneously.
- Efficiently handles extremely large search spaces.
- Avoids local maxima using crossover and mutation.
- Continuously adapts to changing traffic conditions.
- Produces high-quality traffic signal timings.
- Highly scalable for smart city applications.
- Supports real-time optimization and continuous improvement.

Although Simulated Annealing performs better than Hill Climbing, the Genetic Algorithm provides better scalability, adaptability, and robustness for complex smart city traffic management.

Smart City Traffic Signal Optimization System using Artificial Intelligence



QUESTION 3:

i. Explain why this problem requires an online search agent instead of offline search.

Online Search Agent

An **online search agent** makes decisions while interacting with the environment. It does not require complete knowledge of the map before starting its mission.

Why Online Search is Required

- The rover has no complete map of the Martian surface.
- New obstacles are discovered only during exploration.
- It receives limited information from onboard sensors.
- The environment changes as new areas are explored.
- The rover must make immediate navigation decisions without waiting for a full map.

Why Offline Search is Not Suitable

Offline search requires complete knowledge of the environment before planning the route.

For the Mars rover:

- The complete terrain is unknown.
- Hazards cannot be predicted in advance.
- Planning the entire route beforehand is impossible.
- The rover would fail if unexpected obstacles appear.

Therefore, an **online search agent** is the most appropriate choice.

ii. Analyze the challenges of partial observability and dynamic environments.

Partial Observability

The rover can only observe nearby areas using cameras, sensors, and radar.

Challenges

- Hidden obstacles may exist beyond sensor range.
- The rover cannot accurately predict future terrain.
- Limited information increases navigation uncertainty.
- Safe path planning becomes more difficult.

Dynamic Environment

Although Mars changes slowly, the rover continuously discovers new information while moving.

Challenges

- Newly detected rocks or craters require route changes.
- Sensor readings may improve or change over time.
- The rover must frequently update its navigation plan.
- Continuous decision-making is required.

iii. Describe how the rover builds and updates its internal model.

The rover maintains an **internal model (world model)** of its environment.

Working Process

1. Collect sensor data using cameras, LiDAR, and proximity sensors.
2. Detect obstacles, rocks, slopes, and craters.
3. Update the internal map with newly discovered terrain.
4. Mark safe and hazardous regions.
5. Recalculate the safest route.
6. Repeat the process as exploration continues.

Benefits

- Improves navigation accuracy.
- Reduces collision risk.
- Enables adaptive path planning.
- Supports intelligent decision-making.

iv. Compare online search with uninformed and informed search strategies.

Feature	**Online Search**	**Uninformed Search**	**Informed Search**
Environment Knowledge	Unknown	Usually Known	Usually Known
Uses Heuristic	Sometimes	No	Yes
Decision Making	During exploration	Before execution	Before execution
Handles Unknown Terrain	Excellent	Poor	Moderate
Adaptability	Very High	Low	Medium
Suitable for Mars Rover	Excellent	Poor	Moderate

Analysis

- **Uninformed Search** (BFS, DFS) requires a known search space and cannot efficiently handle unknown environments.
- **Informed Search** (Greedy Best-First Search, A*) uses heuristics to improve efficiency but still assumes the environment is mostly known.
- **Online Search** explores, learns, and plans simultaneously, making it ideal for unknown terrains like Mars.

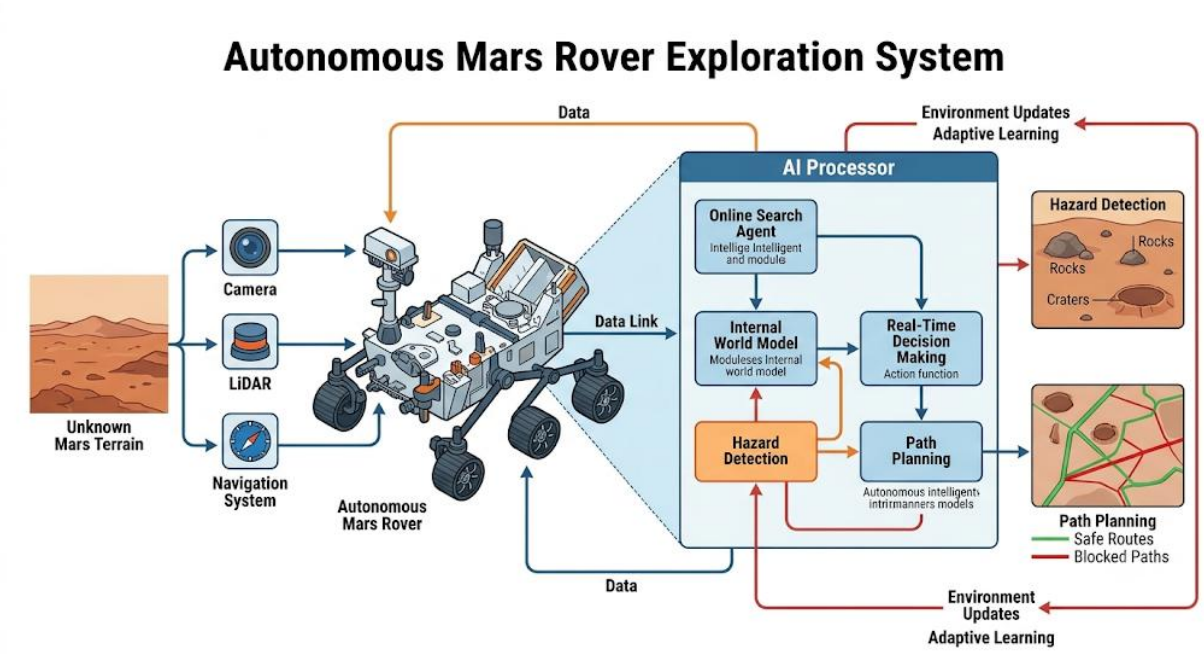
v. Justify the most suitable approach considering uncertainty, adaptability, and safety.

Recommended Approach: Online Search Agent

Justification

An online search agent is the best approach because:

- Operates without a complete map.
- Continuously updates its knowledge during exploration.
- Adapts quickly to newly discovered obstacles.
- Handles uncertainty effectively.
- Uses sensor information to make safe navigation decisions.
- Reduces the risk of collisions with hazards.
- Ensures successful sample collection in unknown environments.
- Supports real-time autonomous decision-making.


Autonomous Mars Rover Exploration System
Data
Environment Updates
Adaptive Learning
AI Processor
Online Search Agent
Intellige Intelligent and module
Internal World Model
Moduleses Internal world model
Real-Time Decision Making
Action function
Hazard Detection
Path Planning
Autonomous intelligent intrirmanners models
Hazard Detection
Rocks
Rocks
Craters
Path Planning
Safe Routes
Blocked Paths
Camera
LiDAR
Navigation System
Unknown Mars Terrain
Autonomous Mars Rover
Data Link
Data
Environment Updates
Adaptive Learning

QUESTION 4:

i. Clearly define variables, domains, and constraints with explanation.

Variables

Variables represent the exams that need to be scheduled.

Examples:

- C1 = Artificial Intelligence
- C2 = Database Management System
- C3 = Operating Systems
- C4 = Computer Networks
- C5 = Data Structures

Each course is considered one variable.

Domains

A domain is the set of possible values that can be assigned to each variable.

For each course, the domain includes:

- Available Time Slots (Morning, Afternoon, Evening)
- Available Exam Halls (Hall A, Hall B, Hall C)

Example:

C1 → {Monday Morning, Monday Afternoon, Tuesday Morning}

Constraints

The timetable must satisfy the following constraints:

1. No Student Exam Overlap

A student enrolled in multiple courses cannot have two exams at the same time.

2. Limited Exam Halls

The number of scheduled exams in a time slot must not exceed the available exam halls.

3. Subject Order Constraint

Certain prerequisite subjects must be conducted before advanced subjects.

Example:

Programming Fundamentals must be scheduled before Artificial Intelligence.

4. Faculty Availability

An exam can only be scheduled when the assigned faculty member is available.

5. Special Accommodation

Students requiring special facilities must be assigned suitable exam halls.

ii. Explain how backtracking search works in this scenario.

Backtracking Search is a systematic search algorithm used to solve CSPs.

It assigns values to variables one by one and checks whether the current assignment satisfies all constraints.

Working Steps

1. Select an unscheduled course.
2. Assign an available time slot and exam hall.
3. Check all constraints.
4. If no constraint is violated, continue scheduling the next course.
5. If a conflict occurs, undo the assignment (backtrack).
6. Try another available time slot.
7. Repeat until all exams are successfully scheduled.

Advantages

- Guarantees a valid timetable if one exists.
- Eliminates invalid schedules.
- Simple and widely used for CSP problems.

Limitations

- May require many backtracking operations.
- Execution time increases as the number of courses grows.

iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency.

Constraint propagation reduces the search space by removing invalid choices before they cause conflicts.

Forward Checking

After assigning a value to one variable, Forward Checking removes inconsistent values from the domains of remaining variables.

Example

Suppose:

Database Management System is scheduled on Monday Morning.

Forward Checking immediately removes Monday Morning from the domains of all courses having common students.

This prevents future conflicts.

Advantages

- Detects conflicts early.
- Reduces unnecessary backtracking.
- Decreases computation time.
- Produces solutions more efficiently.

iv. Analyze real-world challenges and justify the best strategy for scalability.

Real-World Challenges

- Large number of students and courses.
- Limited exam halls.
- Faculty schedule conflicts.
- Last-minute timetable changes.
- Special accommodation requirements.
- Increasing complexity with university size.

Best Strategy

The best approach is to combine:

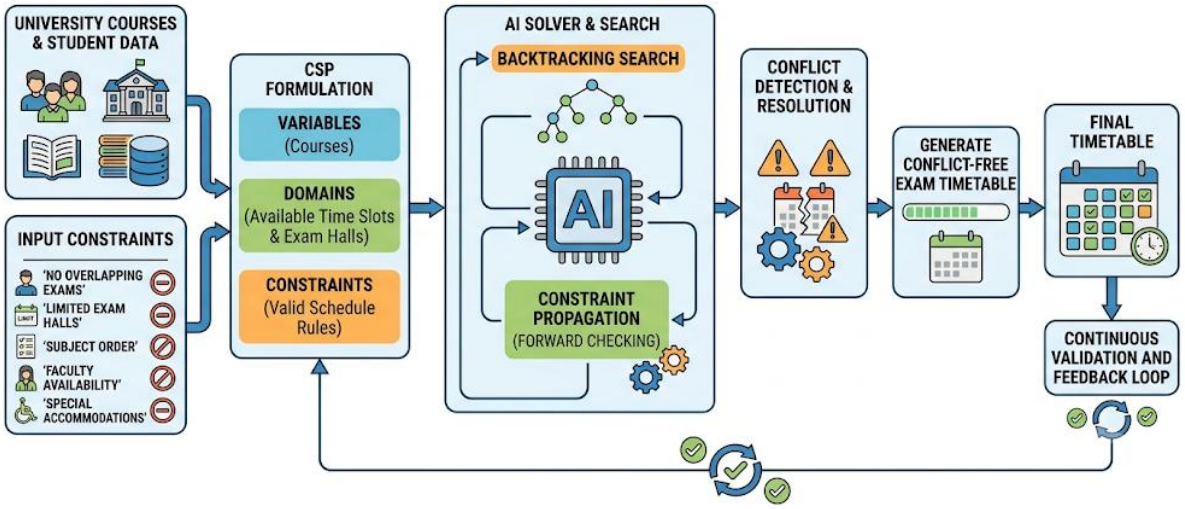
- **Backtracking Search**
- **Constraint Propagation (Forward Checking)**

Justification

- Backtracking guarantees a valid solution.
- Forward Checking reduces the search space.
- Faster timetable generation.
- Efficient handling of multiple constraints.
- Easily scalable for universities with thousands of students and courses.
- Produces conflict-free exam schedules.

AI-based University Exam Timetable Generation System

using **Constraint Satisfaction Problem (CSP)**



QUESTION 5:

i. Explain how the Minimax algorithm models this problem.

The **Minimax algorithm** is an adversarial search algorithm used in two-player games where one player tries to maximize the score while the opponent tries to minimize it.

In this scenario:

- The AI acts as the MAX player, selecting moves that maximize its chances of winning.
- The human player acts as the MIN player, choosing moves that reduce the AI's advantage.
- The AI builds a game tree where each node represents a game state and each branch represents a possible move.
- Minimax evaluates all possible future moves and chooses the move with the best outcome, assuming the opponent also plays optimally.

Working Steps

1. Generate all possible legal moves.
2. Build the game tree.
3. Evaluate terminal game states using an evaluation function.
4. Propagate scores upward through the tree.
5. MAX chooses the highest score.
6. MIN chooses the lowest score.
7. Continue until the best move is selected.

Advantages

- Guarantees the optimal move when the complete game tree is explored.
- Suitable for competitive games.
- Makes intelligent decisions against skilled opponents.

ii. Analyze the computational challenges of large game trees.

In real-time strategy games, the number of possible moves is extremely large.

Challenges

- Every move creates many new possible game states.
- The game tree grows exponentially with each level.
- Searching the complete tree requires significant computation time.

- Large memory is needed to store game states.
- Real-time games require decisions within milliseconds.
- Exploring the full tree may delay the AI's response.

Example

If each position has **10 possible moves**, then:

- Depth 1 → 10 nodes
- Depth 2 → 100 nodes
- Depth 3 → 1,000 nodes
- Depth 5 → 100,000 nodes

This exponential growth is known as the combinatorial explosion, making exhaustive search impractical.

iii. Explain how Alpha-Beta Pruning improves efficiency with detailed reasoning.

Alpha-Beta Pruning is an optimization technique for the Minimax algorithm.

It reduces computation by eliminating branches that cannot influence the final decision.

Working Principle

- **Alpha (α):** Best value that the MAX player can guarantee.
- **Beta (β):** Best value that the MIN player can guarantee.

If at any point **α ≥ β**, the remaining branches are pruned because they cannot produce a better result.

Benefits

- Eliminates unnecessary node evaluations.
- Significantly reduces computation time.
- Allows deeper search within the same time limit.
- Produces the same optimal result as Minimax.
- Improves real-time decision-making.

Example

If the AI already finds a move with a score of **8**, and another branch cannot exceed **8**, that branch is skipped without further evaluation.

iv. Design an evaluation function and justify the factors considered.

An evaluation function estimates the quality of a game state when the search cannot reach the end of the game.

Example Evaluation Function

Evaluation Score =

- (5 × Army Strength)
- (4 × Resource Availability)
- (3 × Territory Control)
- (2 × Defensive Position)
- (2 × Unit Health)
- (4 × Enemy Strength)

Factors Considered

1. Army Strength

More powerful units increase the chance of winning.

2. Resource Availability

More resources allow faster unit production and upgrades.

3. Territory Control

Controlling more strategic locations provides tactical advantages.

4. Defensive Position

Well-defended bases reduce the chance of enemy attacks.

5. Unit Health

Healthy units are more effective during battles.

6. Enemy Strength

A stronger opponent decreases the evaluation score.

These factors help the AI estimate the best move even without exploring the entire game tree.

v. Recommend a strategy for balancing optimality and real-time performance.

Recommended Strategy

The best strategy is to combine:

- Minimax Algorithm
- Alpha-Beta Pruning
- Depth-Limited Search
- Evaluation Function

Justification

- Minimax provides intelligent decision-making.
- Alpha-Beta Pruning removes unnecessary branches.
- Depth-Limited Search ensures decisions are made within the available time.
- The Evaluation Function estimates the quality of non-terminal game states.
- This combination allows the AI to make strong decisions while maintaining real-time performance.